

ICS Lab1 试验报告

魏珏瑾-24307130205

一、测试结果

1.1 规则检查 (check_ops.py)

在终端中执行 `./check_ops.py bits.c`，输出逐题列出当前操作数与上限的比值，最后一行为 `All 19 functions passed operator checks.`，表明全部 19 个函数的运算符合法性与操作数数量均满足约束。

```
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab1/ics-lab1$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 10/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 8/8 operators
oddParity: 12/56 operators
rotateRightBits: 13/16 operators
roundEvenPow2: 20/24 operators
midpointTowardFirst: 21/32 operators
isBetweenEitherOrder: 48/48 operators
mul5Sat: 28/30 operators
classifyAdd3: 35/52 operators
floatScaleThreeHalves: 35/60 operators
floatRoundEven: 27/65 operators
float_i2f: 21/40 operators
bitCount: 29/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

图 1 `./check_ops.py bits.c` 运行结果

1.2 正确性测试 (btest)

在终端中执行 `./btest`，每道题的 Errors 均为 0，最后一行输出 `Total points: 110/110`，全部题目通过正确性验证。

```
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab1/ics-lab1$ ./btest
Score  Rating  Errors  Function
1      1      0      signMask
2      2      0      bitXor
3      3      0      negativePart
4      4      0      copyByteWithin
4      4      0      logicalShift
4      4      0      swapNibblePairs
4      4      0      secondLowestZeroBit
5      5      0      oddParity
5      5      0      rotateRightBits
5      5      0      roundEvenPow2
5      5      0      midpointTowardFirst
7      7      0      isBetweenEitherOrder
7      7      0      mul5Sat
7      7      0      classifyAdd3
7      7      0      floatScaleThreeHalves
10     10     0      floatRoundEven
10     10     0      float_i2f
10     10     0      bitCount
10     10     0      bitReverse
Total points: 110/110
```

图 2 ./btest 运行结果

二、整数题实现思路

2.1 P1 signMask

(1) **题目理解**: 返回仅最高位为 1 的掩码 0x80000000，即符号位掩码。

(2) **思路描述**: 大常量被禁止，故用最小常量 1 经过左移 31 位构造出来，移位量 31 在合法范围内。

2.2 P2 bitXor

(1) **题目理解**: 仅用 ~ 与 & 实现按位异或 $x \oplus y$: 两数对应位不同则该位为 1。

(2) **思路描述**: 先写出 $x \oplus y = (x \& \sim y) | (\sim x \& y)$ ，再用德摩根律（将逻辑运算的等价改写公式，例如非(A 且 B) 等于 (非 A)或(非 B)，用来把禁用的运算换成允许的）把或运算转为与非形式 $a | b = \sim(\sim a \& \sim b)$ ，使表达式中只剩 ~ 映 & 两种运算符。

2.3 P3 negativePart

(1) **题目理解**: x 为负数时返回 -x，否则返回 0，即提取 x 的负数部分。

(2) **思路描述**: 利用算术右移的符号扩展: $x \gg 31$ 在 x 负时为全 1 掩码、非负时为 0。取负用按位取反再加一得到相反数，恒等式 $-x = \sim x + 1$ ；两者按位与即实现按符号选择。

2.4 P4 copyByteWithin

(1) **题目理解**: 把 x 的第 src 个字节复制到第 dst 个字节位置，其余字节不变，字节序号 0-3 从低到高。

(2) **思路描述**: 字节 k 的起始位是 $8k$, 乘法被禁故用左移 $k \ll 3$ 替代; 先取出字节 $(x \gg (\text{src} \ll 3)) \& 0xFF$, 再构造目标字节掩码 $0xFF \ll (\text{dst} \ll 3)$, 将 x 清空目标字节后与对齐后的字节按位或拼接。

2.5 P5 logicalShift

(1) **题目理解**: 对有符号数做高位补 0 的逻辑右移 n 位, $0 \leq n \leq 31$ 。

(2) **思路描述**: C 中 `int` 的 $\gg$ 是算术右移, 高位补符号; 先算术右移, 再用一个高 n 位为 0 的掩码清除符号扩展。掩码由 $((1 \ll 31) \gg n) \ll 1$ 构造后取反得到, $n=0$ 时掩码为全 1, $n=31$ 时只剩最低位, 两个边界均正确。

2.6 P6 swapNibblePairs

(1) **题目理解**: 在每个字节内部交换高 4 位与低 4 位, 字节之间不移动。

(2) **思路描述**: 需要掩码 `0x0F0F0F0F`, 数值较大的常量被禁故用 `0x0F` 倍增构造。低半字节左移 4 位送到高半字节, 高半字节右移 4 位送到低半字节 (算术右移的符号扩展会被掩码清除), 两部分按位或拼接。

2.7 P7 secondLowestZeroBit

(1) **题目理解**: 找出从低到高数第二个 0 位的位置, 返回仅该位为 1 的掩码; 不足两个 0 位时返回 0。

(2) **思路描述**: $\sim x$ 把 0 位变 1 位, 问题化为找 1; 技巧 $a \& (\sim a + 1)$ 即 $a \& -a$ 可孤立出最低的 1; 先孤立并用异或删除最低的 1, 再孤立一次即得第二低的 1。不足两个 0 时中间变量归零, 结果自然为 0。

2.8 P8 oddParity

(1) **题目理解**: x 中 1 的个数为偶数时返回 1, 奇数时返回 0。

(2) **思路描述**: 用分治折叠 (把 32 位逐层对半分、两两合并处理) 异或: $x \wedge (x \gg k)$ 使相距 k 位的两半的奇偶性合并到低半; 依折 16、8、4、2、1, 最终最低位即全部 32 位中 1 个数的奇偶 (奇为 1 偶为 0), 题目要求相反, 所以末尾取逻辑非。算术右移的符号扩展不影响最低位结果。

2.9 P9 rotateRightBits

(1) **题目理解**: 把 x 循环右移 n 位, 低位移出的位绕回高位; n 可达 `INT_MAX`, 实际位数按 $n \bmod 32$ 计。

(2) **思路描述**: 先取 $k = n \& 31$; 右半部分 ($x \gg k$) 用 P5 同款掩码清除符号扩展; 左半部分 $x \ll s$ 其中 $s = 32 - k$, 当 $k = 0$ 时 s 为 32 属非法移位, 故再 $\& 31$ 归零, 此时两半部分均为原值, 结果正确。减法用 $\sim k + 1$ 构造。

2.10 P10 roundEvenPow2

(1) **题目理解**: 把非负 x 舍入到最近的 2^n 倍数, 恰好落在两个候选值正中间的情形时舍入到商为偶数的倍数。

(2) **思路描述**: 商 $q = x \gg n$, 余数 $r = x \wedge (q \ll n)$ (位运算等价于减去低 n 位, 避开禁用的减法与掩码常量)。半程 $half = 2^{(n-1)}$ 用 $1 \ll (n + \sim 0)$ 构造, 再用 $diff = r - half$ 的符号位得到 lt 、 eq , 得出 $gt = !(lt | eq)$ 。进位条件为 gt 或 (eq 且商奇)。

2.11 P11 midpointTowardFirst

(1) **题目理解**: 返回精确数学中点 $(x+y)/2$ 且不允许溢出; 若中点正好落在两个整数中间, 取靠近第一个参数 x 的那个整数。

(2) **思路描述**: 恒等式 $(x+y) + ((x \wedge y) \gg 1)$ 精确等于 $\text{floor}((x+y)/2)$ 且永不溢出 (进位部分与无进位部分分开累加); 和的奇偶由 $(x \wedge y) \& 1$ 给出; 当和为奇数且 $x > y$ 时需向上取整。 $x > y$ 的判断做溢出安全处理: 两数异号时直接看 x 符号, 同号时差不溢出、看差的符号位。

2.12 P12 isBetweenEitherOrder

(1) **题目理解**: 判断 x 是否落在以 a 、 b 为端点的闭区间内, 端点顺序任意, 即 $a \leq x \leq b$ 或 $b \leq x \leq a$ 。

(2) **思路描述**: 核心是一个溢出安全的 $p \geq q$ 判断: 两数异号时只看 p 的符号 (非负必大); 同号时 $p - q$ 不会溢出, 看差的符号位。把同一段溢出安全的 $p \geq q$ 判断的代码, 用四组不同的参数原样展开写四遍 $x \geq a$ 、 $b \geq x$ 、 $x \geq b$ 、 $a \geq x$, 再用 $(c1 \& c2) | (c3 \& c4)$ 覆盖两种端点顺序。为控制操作数, 符号掩码与异号判断只计算一次存入局部变量复用。

2.13 P13 mul5Sat

(1) **题目理解**: 计算 $x * 5$, 溢出时就直接取最大值或最小值封顶: 正溢出返回 INT_MAX , 负溢出返回 INT_MIN 。

(2) **思路描述**: $x * 5 = (x \ll 2) + x$, 但不能只看结果符号 ($x = 0x40000000$ 时结果仍为正但已溢出两次)。正确检测分两级: 一是 $x \ll 2$ 本身丢位 ($(t \gg 2) \wedge x$ 非零, 此时 $|5x| \geq |4x|$ 必溢出); 二是 $4x$ 在界内时最后一次加法的溢出判断 (同号且和符号翻转)。饱和值按 x 符号掩码选择, 用 $\sim \text{ovf} + 1$ 生成的 0/全 1 掩码无分支地完成二选一。

2.14 P14 classifyAdd3

(1) **题目理解**：对三数精确和 $x+y+z$ 分类：大于 INT_MAX 返回 1，小于 INT_MIN 返回 -1，否则 0。

(2) **思路描述**：拆成两次 32 位加法 $s=x+y$ 、 $t=s+z$ ，每次用经典判据检测溢出（同号且和符号翻转，得到全 1/0 掩码 $ov1$ 、 $ov2$ ）。关键在于把每次溢出按方向拆成正、负两组掩码（ $p1/n1$ 用 x 符号、 $p2/n2$ 用 s 符号），并在判定总方向时显式处理两次溢出方向相反、精确和其实未出界的抵消情形（如 $\text{TMax}+\text{TMax}+\text{TMin}$ 精确值为 -2 应返回 0）： $pos=(\sim ov1\&p2)|(p1\&\sim n2)$ ， neg 对称。最后 $(pos\&1)$ 减 $(neg\&1)$ 得到 1/0/-1 三态，其中取负用 $\sim(neg\&1)+1$ 的补码写法。

3.15 P18 bitCount

(1) **题目理解**：统计 x 的 32 位中 1 的个数。

(2) **思路描述**：分治折叠：先把相邻 2 位组内的 1 个数加到本组内（值 0-2），再逐级合并成 4 位组、8 位组（每字节 0-8），最后把分散在各字节里的计数逐个相加汇总成一个数聚到最低字节并 $\&0x3F$ 取出。三个交替掩码 $0x55555555/0x33333333/0x0F0F0F0F$ 均由 $0x55/0x33/0x0F$ 倍增构造，避开大常量禁令。组内计数不会溢出组边界。

2.16 P19 bitReverse

(1) **题目理解**：将 32 位整体倒序：第 i 位与第 $31-i$ 位互换。

(2) **思路描述**：倒序 = 五轮分组交换（按照 1、2、4、8、16 位的大小逐级加倍对折）。操作数预算极紧，关键是用已有的掩码经过移位和异或制造出新掩码，省运算次数的办法：从 $0xFFFF$ 出发，用 $m\wedge(m\ll k)$ 依次异或出 $0x00FF00FF$ 、 $0x0F0F0F0F$ 、 $0x33333333$ 、 $0x55555555$ ，每个只花 2 个操作符，掩码总成本仅 10。最后一步高低 16 位互换时 $(x\gg 16)$ 必须 $\&m16$ ，否则算术右移的符号扩展会污染高半部分。

三、浮点题实现思路

3.1 P15 floatScaleThreeHalves

(1) **题目理解**：返回 $f*3/2$ 的位级表示，舍入到偶数，即正好在中间时选末位是偶数的结果，避免总往一边舍造成偏差；保持 ± 0 符号；参数为 NaN 时原样返回。

(2) **思路描述**：浮点数按 32 位拆为符号、指数、尾数三部分。普通浮点数的最高有效位恒为 1 但不存储，计算前先补上该位，得整数尾数 m ，则精确结果为 $3m\times 2^E$ ；除以 2 不做真除法，等效为指数减 1，与补隐含位的调整合并后取 $E=e-151$ （极小值格式

取 $E=-150$)。m 乘 3 后最多 26 位，超出尾数字段容量，逐次右移至 24 位以内，每次右移同时指数加 1 以保持数值不变；被移出的位不直接丢弃，单独记录最后移出的一位作为舍入判断依据，更早移出的位只要有 1 则置一标志，若之后又有新位移出，原舍入判断位并入该标志。移位完成后做一次舍入：丢弃部分超过一半则进一，恰好一半时取末位为偶数的结果。最后按极小值格式、普通格式、超出范围取无穷大三种类别拼回 32 位输出；指数字段与尾数字段在编码中相邻，进位可直接传递至指数，无需单独处理。题面禁止显式类型转换，指数字段以 int 表达式移位后与符号位按位或拼接。

3.2 P16 floatRoundEven

(1) **题目理解**：把 unsignedfloat 表示的浮点数舍入到最近整数，半程舍到偶数，结果仍以浮点位模式返回；舍入为零时保留符号（返回 -0）；NaN 与无穷大原样返回。

(2) **思路描述**：按阶码分档： $e=255$ 原样返回； $e \geq 150$ 时尾数全为整数位，已是整数； $e \leq 125$ 时绝对值小于 0.5，舍入为 ± 0 ； $e=126$ 时值在 $[0.5,1)$ ，恰为 0.5 时舍到偶数 0、大于 0.5 时进为 1； $127 \leq e \leq 149$ 时令 $\text{shift}=150-e$ ，清掉尾数低 shift 位即向下取整，余数 rem 与半程 half 比较，大于半程或等于半程且整数部分为奇时加 mask+1 进位，进位会自然传递到阶码字段（如 $2^{24-0.5}$ 类边界）。

3.3 P17 float_i2f

(1) **题目理解**：返回 (float)x 的位级表示，需自行完成舍入到偶数。

(2) **思路描述**：先处理 0 与符号（负数取绝对值，INT_MIN 取负回绕后位模式正确）；左移直到最高位到达 bit31，阶码从 $158=127+31$ 随移动递减；移完后 bit30..bit8 作为尾数字段，低 8 位是被丢弃的多余位，按到偶数规则舍入：大于 0x80 必进，恰为 0x80 时仅当尾数为奇时进；舍入进位使尾数达 0x800000 时清零并阶码加一（跨 2 的幂边界）。

四、总结

本实验共完成 19 个函数，规则检查与正确性测试均通过，最终得分 110/110。整数题的核心技巧可归纳为：符号掩码 ($x \gg 31$) 与补码取负 ($\sim x + 1$) 实现无分支的条件选择与比较；异或取消进位 ($a \& -a$ 孤立最低位)；同号相加且符号翻转的溢出判据；分治折叠（奇偶校验、位计数）与掩码倍增/派生（小常量构造大掩码）；以及 32 位位向量模型下的回绕运算。浮点题的关键则是理解尾数、阶码、非规格化数的统一表示，并在纯整数域中用舍入位与粘滞位严格实现舍入到偶数。

调试过程中的主要教训：一是操作数预算紧张的题目（如 `bitReverse 34/34`、`isBetweenEitherOrder 48/48`）必须通过复用局部变量压缩重复子表达式；二是运算符合法不代表结果正确，必须结合 `btest` 的失败样例排查符号、溢出、移位与舍入边界；三是每次修改后必须重新 `make clean` 与 `make all`，否则测试的可能仍是旧代码。